Дисциплина
«Архитектура и программное обеспечение
ЭВМ и систем»

ОТЧЁТ
по лабораторной работе №1

Изучение аппаратно-программной архитектуры
процессоров семейства Intel:
разработка ассемблерной вставки

Вариант 5

Выполнил: Батулин Михаил
Группа: АС-66

2026

1. Цель и постановка задачи

Изучить работу с регистрами процессора, адресацией памяти и строковыми командами x86. Реализовать одинаковое преобразование строк отдельной функцией C++ и функцией с ассемблерной вставкой; экспериментально сравнить их быстродействие.

Вариант 5: «Даны 2 строки. Совместить первую половину строки 1 с первой половиной строки 2». Основание: методические указания, с. 271-272.

Под совмещением понимается последовательное соединение двух первых половин: сначала символы первой строки, затем второй. Если длина нечётная, в первую половину входят floor(n / 2) символов. Центральный символ исключается. Это принятое уточнение, поскольку правило для нечётной длины в задании отдельно не определено.

Пусть n1 и n2 - длины входных строк, k1 = floor(n1 / 2), k2 = floor(n2 / 2). Тогда R = S1[0:k1] + S2[0:k2]. Для результата выделяется k1 + k2 + 1 байт; последний байт содержит нулевой терминатор.

Строка 1	Строка 2	Результат
ABCDEFGH	12345678	ABCD1234
abcde	1234567	ab123
пустая строка	abcd	ab
x	y	пустая строка

2. Среда и представление данных

Проект создан для Microsoft Visual Studio, язык C++17, платформа Win32/x86. Фактическая сборка выполнена MSVC 19.51 (Visual Studio 2026). Для сравнения использована конфигурация Release с /O2. Исходники также проверены в Debug.

Строки обрабатываются как массивы байтов. Консоль переключается на Windows-1251: русские и английские буквы в этой кодировке занимают один байт. Произвольный Unicode и ввод UTF-8 не поддерживаются. Полный ввод выполняет std::getline, поэтому пробелы и пустые строки сохраняются.

3. Алгоритм и работа ассемблерной вставки

Программа считывает две строки, вычисляет k1 и k2, создаёт два независимых выходных буфера и передаёт одинаковые входные данные функциям CombineCpp и CombineAsm. После выполнения сравниваются все байты результатов вместе с терминатором. Входные строки не изменяются.

C++ выполняет два цикла копирования. В ассемблерной функции ESI хранит адрес источника, EDI - текущий адрес назначения, ECX - количество копируемых байтов. Команда CLD сбрасывает флаг направления DF: после MOVSB адреса увеличиваются. Префикс REP повторяет MOVSB ECX раз; при нулевом счётчике копирования нет.

После первой REP MOVSB указатель EDI уже указывает за первой половиной результата. Для второй половины меняются ESI и ECX, а EDI сохраняется. После второго копирования команда MOV BYTE PTR [EDI], 0 записывает терминатор. Сохранение используемых невольтильных регистров обеспечивает компилятор MSVC в прологе и эпилоге функции.

Буферы не должны перекрываться. В этой программе условие обеспечивается отдельными std::vector. Время преобразования O(k1 + k2), дополнительная память внутри каждой функции O(1), память результирующей строки O(k1 + k2).

4. Методика проверки

Набор состоит из восьми фиксированных примеров и 300 воспроизводимых случайных пар строк длиной до 256 байт. Для каждого примера проверяются обе реализации, содержимое результата, терминатор, защитный байт после него и неизменность входных строк. Ожидаемая строка для случайных случаев независимо строится стандартной функцией substr.

Отдельно через консольный интерфейс проверены русские строки в Windows-1251, пробелы, пустые строки и обрыв ввода. Код завершения при корректной работе - 0; при недостающем вводе - 1. Подробный протокол находится в rep/results/verification.txt.

Сборка Release / Win32 и Debug / Win32 завершилась без ошибок и предупреждений. Все 308 случаев прошли для обеих функций. Скриншоты показывают реальные окна консоли, а текстовые протоколы сохранены рядом с отчётом.

5. Результаты выполнения

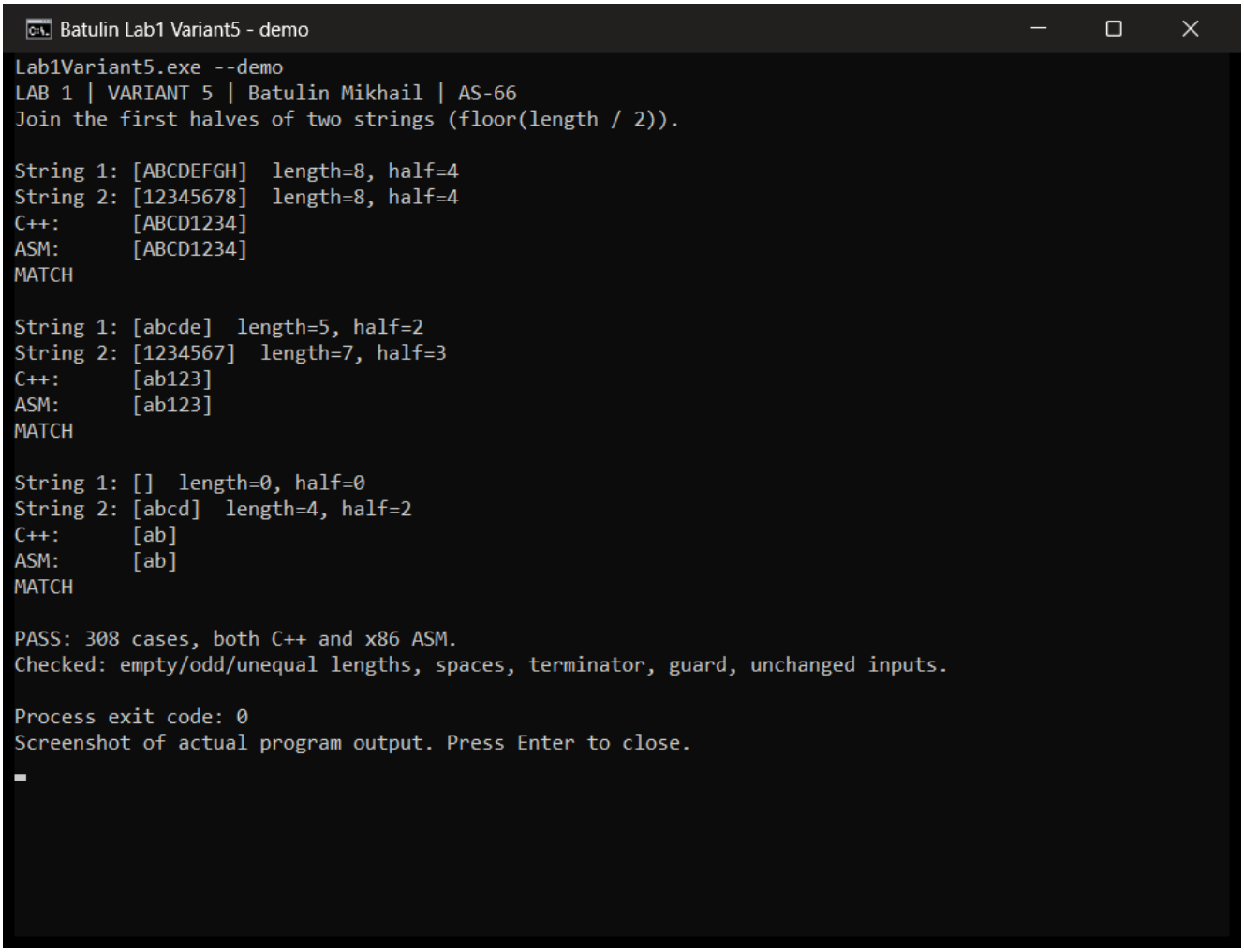


Рисунок 1 - Режим --demo: чётные длины, нечётные длины и пустая первая строка. Для каждой пары обе реализации дают одинаковый результат; затем проходят 308 тестов.

На первом примере в результат входят ABCD и 1234. Для строк abcde и 1234567 длины половин равны двум и трём, поэтому получается ab123. Пустая первая строка не добавляет байтов: результатом третьего примера становится ab.

6. Измерение быстродействия

Замер выполнен 5 ноября 2026 г. на Intel Core i5-12450H. Для каждой длины выполнен прогрев из 1000 вызовов обеих функций, затем девять серий. Порядок C++/ASM чередуется. В таблице показана медиана времени одного вызова для каждой реализации.

Подготовка строк, выделение памяти и вычисление длин находятся вне измеряемого участка. Каждая функция помечена noinline. Во время цикла байт результата добавляется к volatile-контрольной сумме; время включает одинаковые расходы вызова, цикла и подсчёта суммы. Измеряется обработка уже подготовленных строк в памяти.

Байт в каждой строке	Вызовов в серии	C++, нс/вызов	ASM, нс/вызов	C++ / ASM
8	200000	5.78	9.78	0.59
64	100000	7.01	9.54	0.74
1024	20000	32.98	29.23	1.13
16384	2000	94.60	91.50	1.03

Входные строки имеют одинаковую чётную длину. Результат содержит половину каждой, то есть столько же байтов, сколько одна исходная строка. Отношение C++/ASM больше единицы означает преимущество ASM; меньше единицы - преимущество C++.

7. Выводы

Реализовано соединение первых половин двух строк двумя способами. Обе функции дают одинаковые результаты на всех проверенных данных. В данном запуске на строках по 8 и 64 байт C++ быстрее: время составляет 5.78 и 7.01 нс против 9.78 и 9.54 нс для ASM. На строках по 1024 и 16384 байт небольшое преимущество получает REP MOVSB.

Ручная ассемблерная вставка не гарантирует ускорения. Результат зависит от длины блока, оптимизаций C++, реализации строковых команд процессором и нагрузки системы. Разница на длинных строках невелика; по одному компьютеру и одному запуску нельзя делать общий вывод для всех процессоров. Цель работы достигнута: освоены передача адресов в ассемблерную вставку, управление DF и применение REP MOVSB.

Результаты измерения в консоли

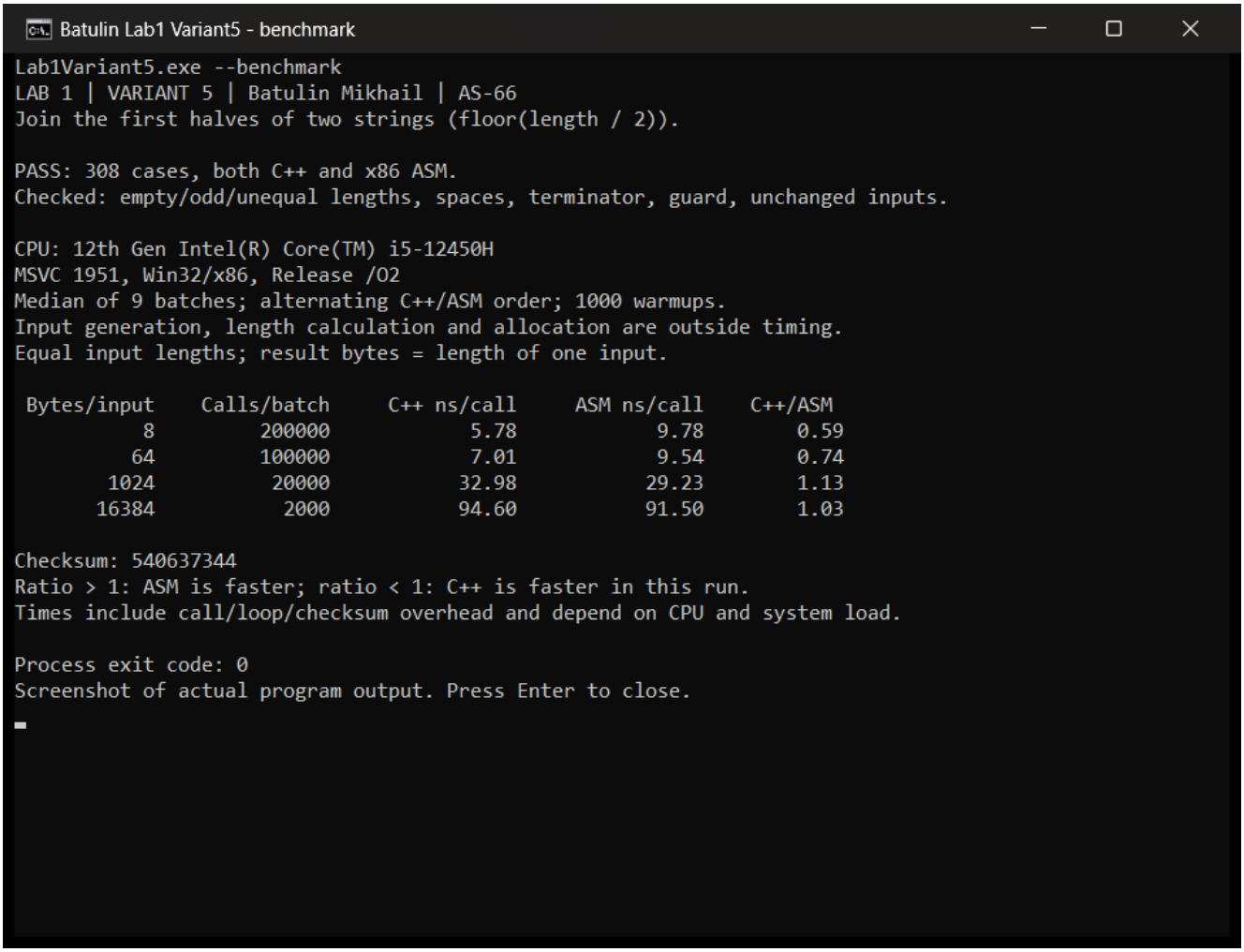


Рисунок 2 - Режим --benchmark: процессор, компилятор, число серий, медианные времена и контрольная сумма фактического запуска.

Воспроизведение

Открыть src/Lab1Variant5.sln, выбрать Release / Win32 и запустить Ctrl+F5. Для готовых примеров выполнить build_run.cmd --demo; для тестов - build_run.cmd --test; для таблицы времени - build_run.cmd --benchmark. Требуется компонент Visual Studio «Разработка классических приложений на C++».

Файлы main.cpp, combine.cpp, combine.h и tests.cpp входят в проект. Приложение ниже содержит полный текст программных файлов. Решение и командный сценарий сборки находятся в src/.

Приложение. combine.h

```
#pragma once
#include <cstddef>

// Buffers must not overlap; out has at least firstCount + secondCount + 1 bytes.
void CombineCpp(const char* first, std::size_t firstCount,
                const char* second, std::size_t secondCount, char* out);
void CombineAsm(const char* first, std::size_t firstCount,
                const char* second, std::size_t secondCount, char* out);
```

Приложение. combine.cpp

```
#include "combine.h"

#if !defined(_MSC_VER) || !defined(_M_IX86)
#error Build with MSVC for Win32 (x86): inline __asm is required.
#endif

__declspec(noinline) void CombineCpp(const char* first, std::size_t firstCount,
                                     const char* second, std::size_t secondCount, char* out) {
    for (std::size_t i = 0; i < firstCount; ++i) out[i] = first[i];
    for (std::size_t i = 0; i < secondCount; ++i) out[firstCount + i] = second[i];
    out[firstCount + secondCount] = '\0';
}

__declspec(noinline) void CombineAsm(const char* first, std::size_t firstCount,
                                     const char* second, std::size_t secondCount, char* destination) {
    __asm {
        cld                         // Forward copying; DF = 0.
        mov esi, first              // Source address for the first half.
        mov edi, destination        // Destination is advanced by MOVSB.
        mov ecx, firstCount
        rep movsb                   // Zero count also works for an empty half.
        mov esi, second
        mov ecx, secondCount
        rep movsb
        mov byte ptr [edi], 0        // C string terminator, excluded from lengths.
    }
}
```

Приложение. main.cpp (1/2)

```cpp
#include "combine.h"
#include <Windows.h>
#include <intrin.h>
#include <algorithm>
#include <array>
#include <chrono>
#include <cstdint>
#include <cstring>
#include <iomanip>
#include <iostream>
#include <limits>
#include <string>
#include <vector>

int RunTests();
using Combine = void (*)(const char*, std::size_t, const char*, std::size_t, char*);

bool ShowResult(const std::string& first, const std::string& second) {
    const auto n1 = first.size() / 2, n2 = second.size() / 2;
    std::vector<char> cpp(n1 + n2 + 1), assembly(n1 + n2 + 1);
    CombineCpp(first.c_str(), n1, second.c_str(), n2, cpp.data());
    CombineAsm(first.c_str(), n1, second.c_str(), n2, assembly.data());
    const bool same = cpp == assembly;
    std::cout << "String 1: [" << first << "]  length=" << first.size() << ", half=" << n1
              << "\nString 2: [" << second << "]  length=" << second.size() << ", half=" << n2
              << "\nC++:      [" << cpp.data() << "]\nASM:      [" << assembly.data() << "]\n"
              << (same ? "MATCH" : "ERROR: results differ") << "\n\n";
    return same;
}

double Measure(Combine fn, const std::string& first, const std::string& second,
               std::vector<char>& out, std::size_t iterations, volatile std::uint64_t& checksum) {
    const auto n1 = first.size() / 2, n2 = second.size() / 2;
    const auto start = std::chrono::steady_clock::now();
    for (std::size_t i = 0; i < iterations; ++i) {
        fn(first.c_str(), n1, second.c_str(), n2, out.data());
        checksum += static_cast<unsigned char>(out[i % (n1 + n2)]);
    }
    const auto stop = std::chrono::steady_clock::now();
    return std::chrono::duration<double, std::nano>(stop - start).count() / iterations;
}

int Benchmark() {
    if (RunTests() != 0) return 1;
    char cpu[49]{};
    int regs[4]{};
    __cpuid(regs, static_cast<int>(0x80000000u));
    if (static_cast<unsigned>(regs[0]) >= 0x80000004u) {
        for (int i = 0; i < 3; ++i) {
            __cpuid(regs, static_cast<int>(0x80000002u + i));
            std::memcpy(cpu + 16 * i, regs, 16);
        }
    }
    std::cout << "\nCPU: " << cpu << "\nMSVC " << _MSC_VER << ", Win32/x86";
#ifdef NDEBUG
    std::cout << ", Release /O2\n";
#else
    std::cout << ", Debug (use Release for timing)\n";
#endif
    std::cout << "Median of 9 batches; alternating C++/ASM order; 1000 warmups.\n"
              << "Input generation, length calculation and allocation are outside timing.\n"
              << "Equal input lengths; result bytes = length of one input.\n\n"
              << " Bytes/input    Calls/batch     C++ ns/call     ASM ns/call    C++/ASM\n";
    volatile std::uint64_t checksum = 0;
    const std::array<std::size_t, 4> lengths{8, 64, 1024, 16384};
    const std::array<std::size_t, 4> counts{200000, 100000, 20000, 2000};
    for (std::size_t row = 0; row < lengths.size(); ++row) {
        std::string first(lengths[row], 'A'), second(lengths[row], 'z');
```

Приложение. main.cpp (2/2)

```
        std::vector<char> out(lengths[row] + 1);
        for (int i = 0; i < 1000; ++i) {
            CombineCpp(first.c_str(), first.size() / 2, second.c_str(), second.size() / 2, out.data());
            CombineAsm(first.c_str(), first.size() / 2, second.c_str(), second.size() / 2, out.data());
        }
        std::array<double, 9> cpp{}, assembly{};
        for (std::size_t round = 0; round < cpp.size(); ++round) {
            if (round % 2 == 0) {
                cpp[round] = Measure(CombineCpp, first, second, out, counts[row], checksum);
                assembly[round] = Measure(CombineAsm, first, second, out, counts[row], checksum);
            } else {
                assembly[round] = Measure(CombineAsm, first, second, out, counts[row], checksum);
                cpp[round] = Measure(CombineCpp, first, second, out, counts[row], checksum);
            }
        }
        std::sort(cpp.begin(), cpp.end());
        std::sort(assembly.begin(), assembly.end());
        std::cout << std::setw(12) << lengths[row] << std::setw(15) << counts[row]
                  << std::fixed << std::setprecision(2) << std::setw(16) << cpp[4]
                  << std::setw(16) << assembly[4] << std::setw(12) << cpp[4] / assembly[4] << '\n';
    }
    std::cout << "\nChecksum: " << checksum << "\n"
              << "Ratio > 1: ASM is faster; ratio < 1: C++ is faster in this run.\n"
              << "Times include call/loop/checksum overhead and depend on CPU and system load.\n";
    return 0;
}

int main(int argc, char* argv[]) {
    // Windows-1251 uses one byte per Russian/English character.
    // This is a byte-string exercise; UTF-8 input is not supported.
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    std::cout << "LAB 1 | VARIANT 5 | Batulin Mikhail | AS-66\n"
              << "Join the first halves of two strings (floor(length / 2)).\n\n";
    const std::string mode = argc > 1 ? argv[1] : "";
    int result = 0;
    if (mode == "--test") result = RunTests();
    else if (mode == "--benchmark") result = Benchmark();
    else if (mode == "--demo") {
        if (!ShowResult("ABCDEFGH", "12345678") ||
            !ShowResult("abcde", "1234567") || !ShowResult("", "abcd")) result = 1;
        if (result == 0) result = RunTests();
    } else if (!mode.empty()) {
        std::cerr << "Usage: Lab1Variant5.exe [--demo|--test|--benchmark] [--hold]\n";
        result = 2;
    } else {
        std::string first, second;
        std::cout << "Input encoding: Windows-1251. Spaces are preserved.\nString 1: ";
        if (!std::getline(std::cin, first)) return 1;
        std::cout << "String 2: ";
        if (!std::getline(std::cin, second)) return 1;
        if (first.size() > static_cast<std::size_t>((std::numeric_limits<int>::max)()) ||
            second.size() > static_cast<std::size_t>((std::numeric_limits<int>::max)())) return 1;
        std::cout << '\n';
        result = ShowResult(first, second) ? 0 : 1;
        if (result == 0) result = Benchmark();
    }
    if (argc > 2 && std::string(argv[2]) == "--hold") {
        std::cout << "\nPress Enter to close." << std::flush;
        std::cin.get();
    }
    return result;
}
```

Приложение. tests.cpp

```
#include "combine.h"
#include <cstring>
#include <iostream>
#include <random>
#include <string>
#include <vector>

// Detects concatenating whole strings, taking the wrong half, rounding up,
// dropping spaces, writing beyond the terminator, or changing either input.
int RunTests() {
    struct Example { const char* first; const char* second; const char* expected; };
    const Example examples[] = {
        {"ABCDEFGH", "12345678", "ABCD1234"},
        {"abcde", "1234567", "ab123"},
        {"", "", ""},
        {"", "abcd", "ab"},
        {"abcd", "", "ab"},
        {"x", "y", ""},
        {"a b c d!", "12 34 56", "a b 12 3"},
        {"AB", "1234567890", "A12345"}
    };
    int passed = 0;
    const auto check = [&](const std::string& first, const std::string& second,
                           const std::string& expected) {
        const auto savedFirst = first, savedSecond = second;
        for (auto fn : {CombineCpp, CombineAsm}) {
            std::vector<char> out(expected.size() + 2, '#');
            fn(first.c_str(), first.size() / 2, second.c_str(), second.size() / 2, out.data());
            if (std::memcmp(out.data(), expected.c_str(), expected.size() + 1) != 0 ||
                out.back() != '#' || first != savedFirst || second != savedSecond) {
                std::cerr << "FAIL: first=[" << first << "] second=[" << second
                          << "] expected=[" << expected << "] actual=[" << out.data() << "]\n";
                return false;
            }
        }
        return true;
    };
    for (const auto& ex : examples) {
        if (!check(ex.first, ex.second, ex.expected)) return 1;
        ++passed;
    }
    std::mt19937 random(5);
    for (int n = 0; n < 300; ++n) {
        std::string first(random() % 257, ' '), second(random() % 257, ' ');
        for (auto& c : first) c = static_cast<char>(32 + random() % 95);
        for (auto& c : second) c = static_cast<char>(32 + random() % 95);
        const auto expected = first.substr(0, first.size() / 2) + second.substr(0, second.size() / 2);
        if (!check(first, second, expected)) return 1;
        ++passed;
    }
    std::cout << "PASS: " << passed << " cases, both C++ and x86 ASM.\n"
              << "Checked: empty/odd/unequal lengths, spaces, terminator, guard, unchanged inputs.\n";
    return 0;
}

#ifdef LAB_TEST_MAIN
int main() { return RunTests(); }
#endif
```